COMP1002 Data Structures and Algorithms: Final Assignment Report

Student Name: Tushar Pirwani

Student ID: 22464293

Unit: COMP1002 - Data Structures and Algorithms

Date of Submission: 27/10/2025(extention)

Table of Contents

1. Introduction / Overview
2. System Design and Module Integration
    - 2.1 Data Structures Used (Justification)
    - 2.2 Module Integration Flowchart/Diagram
3. Module 1: Graph-Based Hospital Navigation
    - 3.1 Implementation Details
    - 3.2 Algorithm Implementation (BFS, DFS, Dijkstra's)
    - 3.3 Complexity Analysis
    - 3.4 Sample Output & Testing
4. Module 2: Hash-Based Patient Lookup
    - 4.1 Implementation Details
    - 4.2 Core Operations & Collision Handling
    - 4.3 Complexity Analysis
    - 4.4 Sample Output & Testing
5. Module 3: Heap-Based Emergency Scheduling
    - 5.1 Implementation Details
    - 5.2 Priority Metric & Integration
    - 5.3 Heap Operations
    - 5.4 Complexity Analysis
    - 5.5 Sample Output & Testing
6. Module 4: Sorting Patient Records
    - 6.1 Implementation Details
    - 6.2 Performance Benchmark Methodology & Results
    - 6.3 Analysis & Discussion of Benchmark Results
    - 6.4 Complexity Analysis
    - 6.5 Sample Output
7. Reflection
8. Limitations & Assumptions
9. Conclusion
10. References

1. Introduction / Overview

The primary objective of this project is to design, implement, and analyse a modular hospital resource management system. Leveraging fundamental data structures and algorithms studied in COMP1002, the system aims to simulate and optimise critical operational aspects within a hospital environment, focusing on enhancing efficiency in navigation, patient data access, emergency case scheduling, and end-of-day record processing.

The system addresses four key functional requirements through distinct modules:

- **Module 1 (Graph Navigation):** Implements a graph representing the hospital layout to facilitate efficient pathfinding and calculate travel times between various departments (e.g., Emergency, ICU, Labs).
- **Module 2 (Patient Lookup):** Utilises a hash table to provide rapid, O(1) expected time complexity for storing and retrieving patient records based on their unique ID, crucial for timely access to information.
- **Module 3 (Emergency Scheduling):** Employs a heap-based priority queue to manage and prioritise emergency cases according to a defined metric combining clinical urgency and estimated treatment time, ensuring the most critical cases are addressed promptly.
- **Module 4 (Record Sorting):** Implements and benchmarks Merge Sort and Quick Sort algorithms to efficiently organise patient records by treatment duration, enabling performance analysis and supporting reporting requirements.

A modular design approach was adopted, structuring the codebase into distinct Python files for enhanced organisation, readability, and maintainability. Core data structures like the doubly linked list (dsa_linked_list.py) and associated stacks/queues (dsa_stacks_queues.py) provide foundational components. Each primary functional module resides in its own file (module1_graph.py, module2_hash_table.py, module3_heap.py, module4_sorts.py). A central application file (hospital_system.py) orchestrates the system, imports necessary modules, handles user interaction via a command-line menu, and executes the benchmarking procedures.

This report details the design choices, implementation specifics, complexity analysis, and testing results for each module. It begins with an overview of the system design and module integration, followed by in-depth sections covering each of the four modules. Finally, the report includes reflections on the development process, discusses limitations and assumptions, and concludes with a summary of the project outcomes.

2. System Design and Module Integration

2.1 Data Structures Used (Justification)

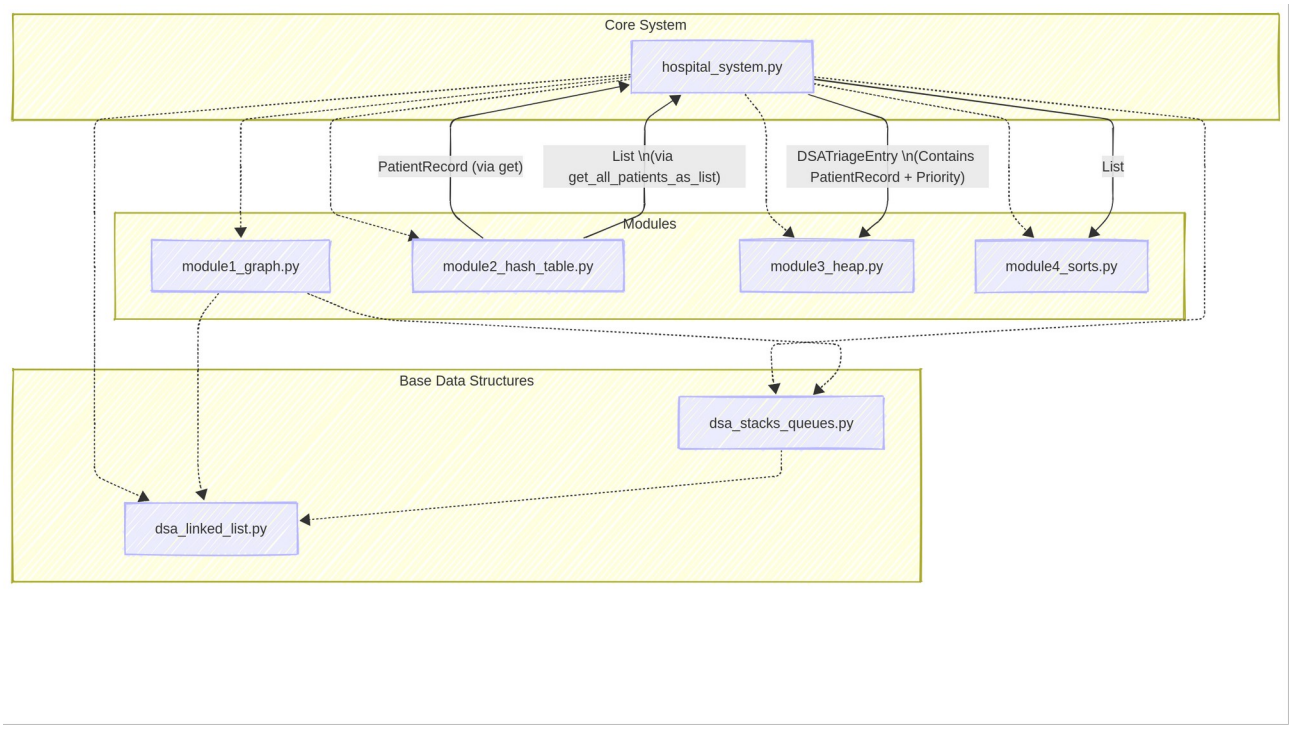
The selection of appropriate data structures is fundamental to achieving the required efficiency for each module's specific task. The following justifications outline the rationale behind the chosen structures:

- **Graph (Adjacency List - Module 1):**
    - **Justification:** A graph is the natural representation for modeling interconnected locations, such as hospital departments connected by corridors. A weighted graph was necessary to represent the travel time (cost) associated with moving between departments. An undirected graph was chosen as corridors are typically bidirectional. An adjacency list representation was implemented using the custom DSALinkedList. This is generally more space-efficient than an adjacency matrix for graphs that are likely to be sparse (where the number of edges `E` is much less than `V^2`), such as a building layout. While adjacency matrix lookups for edge existence are O(1), iterating over neighbors in an adjacency list is often faster for sparse graphs, which benefits traversal algorithms like BFS, DFS, and Dijkstra's.
- **Hash Table (Open Addressing, Linear Probing - Module 2):**
    - **Justification:** The requirement for rapid access to patient records based on a unique Patient ID strongly points towards a hash table, which offers an expected (average) time complexity of O(1) for insertion, deletion, and retrieval. Open Addressing with Linear Probing was selected as the collision resolution strategy. This method avoids the memory overhead and potential complexities of managing separate linked lists (as in chaining) by storing all entries directly within the primary array. While susceptible to primary clustering (where collisions fill consecutive slots), its simplicity and cache-friendliness can be advantageous. Resizing logic (both growing and shrinking based on load factor thresholds) was implemented to maintain performance by keeping the load factor within reasonable bounds, mitigating the negative impacts of clustering and ensuring near O(1) average performance.
- **Heap (Max Heap, Array-Based - Module 3):**
    - **Justification:** A priority queue is needed to efficiently manage emergency cases based on a calculated priority score. A binary heap provides an excellent balance between performance and implementation complexity for this task, offering O(log N) time complexity for both insertion and extraction of the highest-priority item. A Max Heap was specifically chosen because the defined priority formula (Priority = (6 - U) + 1000 / T) yields higher scores for more urgent cases (lower U, shorter T). A Max Heap ensures that the patient with the highest score is always at the root, allowing for O(1) peeking (if needed) and O(log N) extraction. The array-based implementation using numpy is space-efficient and provides good cache locality compared to pointer-based tree structures.
- **Sorting Algorithms (Merge Sort & Quick Sort - Module 4):**

- **Justification:** The task requires sorting patient records for reporting and analysing algorithm performance under different conditions. Implementing two distinct, efficient comparison-based sorting algorithms with an average time complexity of O(N log N) allows for a practical comparison. Merge Sort was chosen for its guaranteed O(N log N) performance regardless of input order and its stability (maintaining relative order of equal elements), although it requires O(N) auxiliary space. Quick Sort was chosen for its typically fast in-place sorting (O(log N) average space complexity due to recursion stack) but acknowledging its potential O(N^2) worst-case time complexity. Implementing Quick Sort with three different pivot strategies (Left, Median-of-Three, Random) allows for direct analysis of how pivot selection impacts performance, particularly demonstrating the failure of the 'Left' pivot on sorted or reversed data, thereby highlighting the practical importance of robust pivot selection.

2.2 Module Integration Flowchart/Diagram

- **Diagram:**



- **Explanation:** The system employs a modular design run by the main hospital_system.py script. User interactions via the menu trigger functions within specific modules. Foundational data structures, DSALinkedList (from dsa_linked_list.py), DSAStack, and DSAQueue (from dsa_stacks_queues.py), provide reusable components imported by relevant modules. hospital_system.py initializes instances of DSAGraph (from module1_graph.py), DSAHashTable (from module2_hash_table.py), and DSAHeap (from module3_heap.py). Graph operations (Menu 1-5) directly call methods within the DSAGraph instance. Patient lookup operations (Menu 10-14) interact with the DSAHashTable instance, which stores PatientRecord objects (also defined in module2_hash_table.py). Adding a patient to triage (Menu 20) involves retrieving the PatientRecord from the DSAHashTable (Module 2), calculating the priority score, creating a

DSATriageEntry (defined in module3_heap.py), and inserting it into the DSAHeap (Module 3). Extracting the next patient (Menu 21) uses the DSAHeap. Sorting operations (Menu 30-31) and the performance benchmark (Menu 32) first retrieve a list of all current PatientRecord objects from the DSAHashTable (using get_all_patients_as_list). This list is then passed to the mergeSort or quickSort functions imported from module4_sorts.py.

3. Module 1: Graph-Based Hospital Navigation

3.1 Implementation Details

The hospital layout and connectivity are modelled using a graph data structure implemented in module1_graph.py. This module provides the foundation for navigational queries within the simulated hospital environment. Two primary classes are defined:

- **DSAGraphNode:** Represents a vertex (department) within the graph. Each node encapsulates:
    - **_label:** A string identifier for the department (e.g., 'Emergency', 'ICU').
    - **_links:** An instance of DSALinkedList serving as the adjacency list. Each element stored in this list is a tuple (neighbour_node, weight), where neighbour_node is a reference to another DSAGraphNode object and weight represents the travel time (cost) along the edge connecting them.
    - **Traversal Attributes:** Several attributes (_visited, _distance, _previous, _in_path) are included to support the implementation of graph traversal and pathfinding algorithms like BFS, DFS, and Dijkstra's. These attributes are managed and reset by the algorithm methods within the DSAGraph class.
- **DSAGraph:** Manages the overall graph structure, containing the collection of all DSAGraphNode objects.
    - **_vertices:** An instance of DSALinkedList storing all the DSAGraphNode objects currently present in the graph.
    - **Core Methods:** Provides essential graph manipulation methods:
        - **add_vertex(label):** Adds a new department to the graph if it doesn't already exist.
        - **add_edge(label1, label2, weight)**: Adds a weighted, undirected edge between two existing vertices. This is achieved by adding the connection to the adjacency lists of both vertices involved, ensuring symmetry.
        - **get_vertex(label):** Retrieves a DSAGraphNode object based on its label.
    - **Helper Methods:** Includes utility functions like _clear_all_visited (to reset node states before traversals) and _sort_neighbors (to ensure consistent, alphabetical processing of neighbours during traversals).
    - **Display Methods:** display_as_list and display_as_matrix provide ways to visualize the graph structure for verification, showing connections and weights.

This graph implementation relies on the custom DSALinkedList (dsa_linked_list.py) for managing the list of vertices and each vertex's adjacency list, adhering to the requirement of building structures from first principles. The graph structure concepts are adapted from COMP1002 Practical 6.

3.2 Algorithm Implementation (BFS, DFS, Dijkstra's)

The DSAGraph class incorporates implementations of the required graph algorithms, developed without reliance on built-in graph libraries:

- **Breadth-First Search (BFS):**
    - **Method:** breadth_first_search_by_level(start_label)
    - **Purpose:** To find all reachable departments from a starting point and group them by the minimum number of corridors (hops) needed to reach them.
    - **Logic:** Implements a standard BFS using a DSAQueue (from dsa_stacks_queues.py). It starts at the source vertex, marks it as visited, and enqueues it along with its level (0). The algorithm then iteratively dequeues a vertex, processes it (adds to the output for its level), and enqueues all its unvisited neighbours with an incremented level. A helper structure (implemented using parallel lists simulating a dictionary) stores the nodes found at each level before printing the grouped results. The _visited flag on each DSAGraphNode prevents cycles and redundant processing. Neighbours are processed in alphabetical order due to the use of get_adjacent_sorted.
- **Depth-First Search (DFS) for Cycle Detection:**
    - **Method:** depth_first_search_cycle_detect(start_label) (calls _dfs_cycle_helper)
    - **Purpose:** To determine if any cycles exist within the graph reachable from the starting department.
    - **Logic:** Uses a recursive helper function (_dfs_cycle_helper). It employs two boolean flags on each DSAGraphNode:
        - **_visited:** Marks nodes that have been visited at any point during the DFS traversal (gray or black in conceptual terms).
        - **_in_path:** Marks nodes that are currently part of the active recursion stack (gray). This flag is set to True when entering the recursion for a node and set back to False when backtracking.
    - **Cycle Detection:** A cycle is detected if the DFS explores an edge leading to a neighbour that is already marked as _in_path. This indicates a "back edge" to an ancestor node currently in the recursion stack, confirming a cycle. The algorithm reports the detection and the specific back edge involved.
- **Dijkstra's Shortest Path Algorithm:**
    - **Method:** find_shortest_path(start_label, end_label)
    - **Purpose:** To find the path with the minimum total travel time (sum of edge weights) between a specified start and end department.
    - **Logic:** Implements Dijkstra's algorithm from first principles.
        - **Initialization***:* Resets all node distances (_distance) to infinity (float('inf')) and _previous pointers to None using _clear_all_visited. Sets the start node's

distance to 0. A standard Python list (unvisited_nodes) is used as a simple substitute for a min-priority queue to hold nodes yet to be finalized.

- **Iteration:** While the unvisited_nodes list is not empty, the algorithm repeatedly selects the node within this list that has the smallest current _distance (found via a linear scan using _find_min_distance_node).
- **Node Finalization:** The selected node (initially the start node) is conceptually "finalized" (removed from the unvisited_nodes list in this implementation).
- **Edge Relaxation:** For the selected node, the algorithm examines each neighbour. If a shorter path to the neighbour is found by going through the current node (current_node.distance + edge_weight < neighbour.distance), the neighbour's _distance is updated, and its _previous pointer is set to the current node.
- **Termination:** The process continues until the destination node is selected or until the smallest distance among remaining unvisited nodes is infinity (meaning any remaining nodes are unreachable from the start).
- **Path Reconstruction:** If the destination node was reached (its distance is not infinity), the shortest path is reconstructed by starting at the end node and following the _previous pointers back to the start node. The path is stored temporarily and then reversed to display the correct order from start to end, along with the calculated total _distance.

- **Citation:** The implementation approach is referenced from Wikipedia's description of Dijkstra's algorithm.

3.3 Complexity Analysis

Let `V` be the number of vertices (departments) and `E` be the number of edges (corridors) in the graph.

- **Space Complexity (Adjacency List):** The space required to store the graph is O(V + E). This includes space for the vertex list and the adjacency lists associated with each vertex.
- **add_vertex: O(V)** in the worst case. This is dominated by the need to check if the vertex label already exists by traversing the _vertices linked list.
- **`add_edge`: O(V)** in the worst case. This is primarily due to the get_vertex calls needed to find the node objects for the given labels, each potentially taking O(V) time. The insertion into each adjacency list (DSALinkedList.insertLast) is O(1).
- **BFS (breadth_first_search_by_level): O(V + E)**. In the worst case, every vertex and every edge will be visited exactly once. The space complexity is O(V) to store the queue, the visited status, and the level groupings.
- **DFS (depth_first_search_cycle_detect): O(V + E)**. Similar to BFS, each vertex and edge is visited at most once during the traversal. The space complexity is O(V) in the worst case for the recursion stack depth (for a path graph) and storing visited/in_path statuses.

- **Dijkstra's (find_shortest_path using List Scan): O(V^2)**. The main loop runs up to V times (once for each vertex). Inside the loop, selecting the minimum distance node (_find_min_distance_node) requires scanning the list of unvisited nodes, taking O(V) time. Relaxing edges connected to the selected node takes, in total across all iterations, O(E) time. The dominant factor is the V iterations multiplied by the O(V) scan for the minimum distance node, resulting in O(V^2). The space complexity is O(V) for storing distances, previous pointers, and the list of unvisited nodes. *(*Note: This complexity arises because a simple list scan is used instead of a more efficient min-priority queue data structure like a binary heap, which would typically yield O(E log V) or O(E + V log V)).

3.4 Sample Output & Testing

The graph module was tested using the sample hospital layout defined during initialization in hospital_system.py. This setup includes 9 vertices (Emergency, ICU, Pharmacy, Radiology, Labs, Operating Theatres, Wards, Outpatient, and the isolated Admin) and 11 weighted, undirected edges, including connections that form cycles.

- **Adjacency List Verification (Menu Option 1):**

```
Enter your choice: 1

--- Adjacency List (Weighted) ---
Emergency: ICU (2.0), Radiology (3.0), Operating Theatres (4.0)
ICU: Emergency (2.0), Operating Theatres (2.0), Wards (5.0)
Pharmacy: Labs (1.0), Outpatient (3.0)
Radiology: Emergency (3.0), Labs (2.0)
Labs: Radiology (2.0), Pharmacy (1.0), Wards (5.0)
Operating Theatres: Emergency (4.0), ICU (2.0), Wards (3.0)
Wards: ICU (5.0), Operating Theatres (3.0), Outpatient (4.0), Labs (5.0)
Outpatient: Wards (4.0), Pharmacy (3.0)
Admin:
----------------------------------
```

- **BFS Test (Menu Option 3, Start: Emergency):**

```
Enter your choice: 3
Enter start department for BFS: Emergency

--- BFS by Level (Start: Emergency) ---
Level 0: Emergency
Level 1: ICU, Operating Theatres, Radiology
Level 2: Wards, Labs
Level 3: Outpatient, Pharmacy
-------------------------------------
```

- **DFS Cycle Detection Test (Menu Option 4, Start: Labs):**

```
Enter your choice: 4
Enter start department for DFS cycle check: Labs

--- DFS Cycle Detection (Start: Labs) ---
Cycle Detected! Path leads back to Labs.
  (Back edge from Pharmacy to Labs)
-----------------------------------------
```

- **Dijkstra's Shortest Path Test (Menu Option 5, Start: Emergency, End: Pharmacy):**

```
Enter your choice: 5
Enter start department: Emergency
Enter destination department: Pharmacy

--- Shortest Path (Dijkstra's) ---
From: Emergency To: Pharmacy
Total time: 6.0 minutes
Path: Emergency -> Radiology -> Labs -> Pharmacy
-----------------------------------
```

- **Isolated Node Test:** Running traversals (BFS, DFS) or shortest path calculations involving the 'Admin' department correctly shows it has no outgoing edges and cannot reach (or be reached by, in the case of Dijkstra's from other nodes) any other part of the graph, confirming the handling of disconnected components.

```
 0. Exit
Enter your choice: 5
Enter start department: Emergency
Enter destination department: Admin

--- Shortest Path (Dijkstra's) ---
From: Emergency To: Admin
No path found from Emergency to Admin.
-----------------------------------
```

These tests collectively demonstrate the correct implementation of the graph structure and the required navigation and analysis algorithms as specified for Module 1.

4. Module 2: Hash-Based Patient Lookup

4.1 Implementation Details

Patient record management utilises a hash table implemented in module2_hash_table.py. This module ensures efficient storage and retrieval of patient data, aiming for O(1) average time complexity for core operations.

- **PatientRecord Class:** A container class designed to hold patient information relevant to the system. It includes fields for patient_id (used as the key in the hash table), name, urgency (used by Module 3), condition, and treatment_duration (set by Module 3, used by Module 4). The class includes a __str__ method for readable output.
- **DSAHashEntry Class:** An internal helper class representing a single slot within the hash table's underlying array. Each entry stores:
    - **key:** The patient ID.
    - **value:** The corresponding PatientRecord object.
    - **state:** An integer flag indicating the slot's status: 0 (FREE - never used), 1 (USED - currently occupied), or -1 (PREVIOUSLY_USED - formerly occupied, now deleted). This state management is crucial for the chosen collision resolution strategy.
- **DSAHashTable Class:** The main class implementing the hash table logic.

- **Storage:** A numpy array (hash_array) is used as the primary storage mechanism, holding DSAHashEntry objects. The size of this array (capacity) is dynamically managed.
- **Collision Resolution:** Implements Open Addressing with Linear Probing. When a collision occurs (the calculated hash index is already occupied), the algorithm probes the next sequential slot (index + 1, wrapping around using the modulo operator) until an available slot is found. The step size is fixed at 1 (_step_hash returns 1).
- **Hash Function:** Employs a polynomial hash function (_hash) based on the ASCII values of the characters in the patient_id string, combined with a prime multiplier (31), modulo the table capacity. This aims to distribute keys across the array indices.
- **Prime Sizing:** The internal array capacity is always maintained as a prime number. Helper methods _find_next_prime and _is_prime ensure that when the table is initialized or resized, the new capacity is the smallest prime number greater than or equal to the target size. Using prime capacities helps reduce collisions, particularly with simple modulo-based hashing and linear probing.
- **Dynamic Resizing:** The hash table automatically resizes to maintain efficiency.
    - **Growth:** If the load factor (count / capacity) reaches the UPPER_THRESHOLD (0.7) before an insertion, the _resize(is_growing=True) method is triggered. This typically doubles the capacity (finding the next prime) and re-hashes all existing elements into the new, larger array.
    - **Shrinking:** If the load factor falls below the LOWER_THRESHOLD (0.3) after a deletion (and the current capacity is greater than the INITIAL_CAPACITY), the _resize(is_growing=False) method is triggered. This typically halves the capacity (finding the next prime, but not going below the initial size) and re-hashes the elements.
- **File Loading:** The read_patients_csv method provides functionality to populate the hash table from a specified CSV file, parsing patient details and inserting them. Basic error handling for file not found and malformed rows is included.

This implementation adapts hashing concepts from COMP1002 Practical 7, specifically modified to use Linear Probing as required.

4.2 Core Operations & Collision Handling

The DSAHashTable class provides the essential operations required for patient lookup:

- **insert(key, value):**
    - Checks load factor and triggers resizing if UPPER_THRESHOLD is met.
    - Calculates the initial hash index using _hash(key).
    - Probes linearly (index = (index + step) % capacity) starting from the initial index.
    - During probing:

        - If a USED slot (1) with the same key is found, it rejects the insertion (prints an error, returns).
        - If a FREE slot (0) or PREVIOUSLY_USED slot (-1) is found, the new key (PatientID) and value (PatientRecord) are placed in that slot (preferring the first encountered PREVIOUSLY_USED slot if one was seen before a FREE slot). The slot's state is set to USED (1), and the count is incremented.
    - If the probing loops back to the start index without finding an available slot (highly unlikely with resizing), an error is indicated.
- **get(key):**
    - Calculates the initial hash index.
    - Probes linearly starting from the initial index.
    - During probing:
        - If a USED slot (1) with a matching key is found, the corresponding value (PatientRecord) is returned.
        - If a FREE slot (0) is encountered, the search stops immediately, as the key cannot exist further down the probe sequence. A KeyError is raised.
        - If a PREVIOUSLY_USED slot (-1) is encountered, the probing *continues*, as the desired key might have been inserted later and placed beyond this deleted slot.
    - If the probing loops back to the start index without finding the key, a KeyError is raised.
- **delete(key):**
    - Calculates the initial hash index.
    - Probes linearly, similar to get, searching for a USED slot (1) with the matching `key`.
    - If a FREE slot (0) is encountered, the key doesn't exist, and KeyError is raised. Probing continues past PREVIOUSLY_USED slots (-1).
    - If the key is found in a USED slot:
        - The value is stored temporarily to be returned.
        - The slot's state is set to PREVIOUSLY_USED (-1) – this is the "tombstone" mechanism crucial for linear probing.
        - The key and value in the slot are cleared (set to `None`).
        - The table's count is decremented.
    - Checks the load factor and triggers shrinking resize if LOWER_THRESHOLD is met.
    - Returns the value that was deleted.
    - If the key is not found after probing, KeyError is raised.

- **Collision Handling Demonstration:** Linear probing handles collisions. When insert calculates an index that is already in state 1 (USED) with a different key, it simply increments the index (modulo capacity) and checks the next slot. This continues until a slot with state 0 or -1 is found. The get and delete operations follow the same probing sequence, continuing past state -1 slots until either the key is found (state 1) or an empty slot (state 0) definitively indicates the key is absent. A practical example occurred during the loading of 20 patients into the initially small table (size 11, then 23), forcing many insertions to probe past occupied slots.

4.3 Complexity Analysis

Let N be the number of items currently stored in the hash table, and M be the current capacity of the underlying array. The load factor is $\alpha = N/M$.

- **Space Complexity: O(M)**. The primary space usage is the array itself. Since the resizing strategy aims to keep M roughly proportional to N (maintaining α within bounds, e.g., $0.3 < \alpha < 0.7$), the space complexity can be considered O(N) in an amortized sense.
- **insert, get, delete (Average Case): O(1)** (Amortized). In the average case, assuming a reasonably uniform hash function and a load factor kept well below 1 by resizing, the expected number of probes required to find an element or an empty slot is constant. While resizing itself takes O(N) time, it happens infrequently enough that its cost, when averaged over many O(1) operations, does not change the overall amortized average complexity from O(1).
- **insert, get, delete (Worst Case): O(N)**. The worst case occurs when collisions cause significant clustering (primary clustering in the case of linear probing), forcing the algorithm to probe through a large fraction, or even all, of the occupied slots. This can happen if the hash function distributes keys poorly or if the load factor becomes very high (approaching 1).
- **Resizing (_resize): O(M_old + N) ≈ O(N)**. Creating the new array takes time proportional to the new size (which is proportional to N). Re-inserting the N existing elements into the new table takes, on average, O(1) time per element, totalling O(N). Therefore, the overall resizing operation is O(N).

4.4 Sample Output & Testing

The hash table module was tested through interaction via the main menu, including loading initial data, adding, searching, and deleting records.

- **Initial Load & Resizing (Program Start):**

```
Loading patients from patients.csv... Header: ['PatientID', 'Name', 'Urgency', 'Condition']
Resizing hash table from 11 to 23...
Resize complete. Current count: 8
Resizing hash table from 23 to 47...
Resize complete. Current count: 17
Successfully loaded 20 patients.
```

- **Display Table after Load (Menu Option 13):**

```
Enter your choice: 13

--- Current Patients in System (Hash Table) ---
  Index 9: ID: P10010, Name: Laura Thomas, Urgency: 9, Condition: Allergic Reaction, Duration: 0 mins
  Index 10: ID: P10011, Name: Robert Garcia, Urgency: 4, Condition: Fever, Duration: 0 mins
  Index 11: ID: P10012, Name: Linda Rodriguez, Urgency: 7, Condition: Back Pain, Duration: 0 mins
  Index 12: ID: P10013, Name: James Hernandez, Urgency: 9, Condition: Concussion, Duration: 0 mins
  Index 13: ID: P10014, Name: Maria Lopez, Urgency: 5, Condition: Stomach Flu, Duration: 0 mins
  Index 14: ID: P10015, Name: William Gonzalez, Urgency: 10, Condition: Seizure, Duration: 0 mins
  Index 15: ID: P10016, Name: Karen Moore, Urgency: 8, Condition: Pneumonia, Duration: 0 mins
  Index 16: ID: P10017, Name: Paul Allen, Urgency: 7, Condition: Kidney Stone, Duration: 0 mins
  Index 17: ID: P10018, Name: Nancy Young, Urgency: 10, Condition: Stroke, Duration: 0 mins
  Index 18: ID: P10019, Name: Mark King, Urgency: 5, Condition: Concussion, Duration: 0 mins
  Index 26: ID: P10001, Name: John Smith, Urgency: 8, Condition: Broken Arm, Duration: 0 mins
  Index 27: ID: P10002, Name: Sarah Jones, Urgency: 10, Condition: Chest Pains, Duration: 0 mins
  Index 28: ID: P10003, Name: David Lee, Urgency: 5, Condition: Sprained Ankle, Duration: 0 mins
  Index 29: ID: P10004, Name: Emily White, Urgency: 9, Condition: Severe Burn, Duration: 0 mins
  Index 30: ID: P10005, Name: Michael Brown, Urgency: 7, Condition: Laceration, Duration: 0 mins
  Index 31: ID: P10006, Name: Jessica Davis, Urgency: 3, Condition: Common Cold, Duration: 0 mins
  Index 32: ID: P10007, Name: Chris Wilson, Urgency: 10, Condition: Difficulty Breathing, Duration: 0 mins
  Index 33: ID: P10008, Name: Amanda Taylor, Urgency: 6, Condition: Migraine, Duration: 0 mins
  Index 34: ID: P10009, Name: Daniel Martinez, Urgency: 8, Condition: Appendicitis, Duration: 0 mins
  Index 40: ID: P10020, Name: Betty Wright, Urgency: 9, Condition: Sepsis, Duration: 0 mins
Total Patients: 20 | Table Capacity: 47 | Load Factor: 0.43
------------------------------------------------
```

- **Insert Operation (Menu Option 10):**

```
0. Exit
Enter your choice: 10
--- Add New Patient ---
Enter Patient ID: P1
Enter Patient Name: Test Patient
Enter Urgency (1=High, 5=Low): 4
Enter Condition: breathing issues
```

- **Search Operation – Hit (Menu Option 11):**

```
Enter your choice: 11
--- Find Patient ---
Enter Patient ID to find: P10001
Found: ID: P10001, Name: John Smith, Urgency: 8, Condition: Broken Arm, Duration: 0 mins
```

- **Search Operation - Miss (Menu Option 11):**

```
Enter your choice: 11
--- Find Patient ---
Enter Patient ID to find: P999
"Key 'P999' not found in hash table."
```

- **Delete Operation (Menu Option 12):**

```
Enter your choice: 12
--- Delete Patient ---
Enter Patient ID to delete: P10001
Deleted: ID: P10001, Name: John Smith, Urgency: 8, Condition: Broken Arm, Duration: 0 mins
```

Collision Handling Explanation: While direct step-by-step probing is not logged, the successful loading of 20 diverse patient IDs into a table that started at size 11 inherently demonstrates collision handling. Keys like P10001 and P10010 (or others) likely hashed to conflicting initial indices. The linear probing mechanism correctly resolved these by placing colliding records in the next available slots (state 0 or -1). The ability to subsequently get and delete all records confirms that the probing sequence used during insertion is correctly followed during search and deletion, including skipping over tombstone slots (state -1). The resizing operations further demonstrate collision management by rehashing elements into larger arrays, redistributing them to potentially reduce clustering.

These tests validate the implementation of the hash table, including its core operations, dynamic resizing, file loading, and the implicit success of the linear probing collision handling strategy.

5. Module 3: Heap-Based Emergency Scheduling

5.1 Implementation Details

Emergency case scheduling is handled by a Max Heap priority queue implemented in module3_heap.py. This structure ensures that the patient deemed highest priority according to the defined metric is always readily available for extraction.

- **DSATriageEntry Class:** A container class designed to hold the elements within the heap. Each entry stores:
    - **priority_score:** A floating-point number calculated based on patient urgency and treatment time. Higher scores indicate higher priority.
    - **patient_record:** A reference to the corresponding PatientRecord object (from Module 2) associated with this triage request. The class includes a __str__ method for readable output, formatting the priority score.

- **DSAHeap Class:** The main class implementing the Max Heap logic using an array.
    - **Storage:** A numpy array (heap) is used as the underlying storage, holding DSATriageEntry objects. The heap structure is maintained implicitly using array indices (parent at (i-1)//2, children at 2i+1 and 2i+2).
    - **Heap Property:** Maintains the Max Heap property, where the priority_score of any parent node is greater than or equal to the priority_score of its children. This ensures the highest priority entry is always at the root (index 0).
    - **Core Operations:** Implements insert(entry) to add a new triage request and extract_next() to remove and return the highest priority request.
    - **Heap Maintenance:** Relies on two private helper methods to maintain the heap property after modifications:
        - _trickle_up(index): Called after insert. Compares the newly inserted element at index with its parent. If the child's priority is higher, they are swapped, and the process continues recursively (or iteratively in this implementation) up towards the root.
        - _trickle_down(index): Called after extract_next. Compares the element at index (typically the new root) with its children. If either child has a higher priority, the element is swapped with the *largest* of its children, and the process continues recursively (or iteratively) down the heap.
    - **Helper Methods:** Includes is_empty(), is_full(), and get_count() for status checks.
    - **Tracing:** Incorporates print statements within insert, extract_next, _trickle_up, and _trickle_down to log the operations being performed and specifically show which elements are being swapped during the heap maintenance processes (trickling). This fulfills the requirement for observability.

This implementation adapts heap concepts from COMP1002 Practical 8, specifically using a Max Heap as recommended for the priority scoring method.

5.2 Priority Metric & Integration

- **Priority Metric:** The system calculates the priority score for each triage request using the precise formula mandated by the assignment specifications: priority_score = (6 - patient.urgency) + (1000 / treatment_time) Where:
    - patient.urgency (U) is retrieved from the PatientRecord object (obtained from Module 2). This value must be within the range 1-5 (inclusive), where 1 represents the highest clinical urgency. The (6 - U) term effectively inverts this, giving a higher numerical value (5) to the most urgent patients (U=1).
    - treatment_time (T) is the expected duration of treatment in minutes, provided by the user during the triage addition process (Menu Option 20). This value must be positive (T > 0) to avoid division by zero and ensure the formula component 1000 / T correctly contributes higher priority to shorter treatments. This composite score ensures that both clinical urgency and resource time (treatment duration) factor into the final prioritization, with higher scores representing higher overall priority.

- **Integration (Module 2 -> Module 3):** The process of adding a patient to the triage queue (handled in hospital_system.py, Menu Option 20) demonstrates the integration:
    - The PatientRecord is fetched from the DSAHashTable (Module 2) using the patient ID.
    - The patient's urgency is validated (must be 1-5).
    - The treatment_time is obtained from user input and validated (must be > 0).
    - The treatment_duration attribute of the PatientRecord object (still referenced from the hash table) is updated with this time. This crucial step makes the duration available for Module 4 sorting.
    - The priority_score is calculated using the validated urgency and treatment time.
    - A DSATriageEntry object is created containing the calculated score and the PatientRecord.
    - This DSATriageEntry is inserted into the DSAHeap (Module 3) using its insert method.

5.3 Heap Operations

The DSAHeap class implements the two fundamental priority queue operations:

- **insert(entry):** This method adds a new DSATriageEntry to the heap.
    1. It first checks if the heap is full; if so, it raises an OverflowError.
    2. The new entry is placed at the next available position in the underlying array (index count).
    3. The count is incremented.
    4. The _trickle_up method is called starting from the index of the newly added element (count - 1). This method compares the element with its parent and swaps them if the child has a higher priority score, continuing this process up the heap until the Max Heap property is restored or the root is reached.
- **extract_next():** This method removes and returns the DSATriageEntry with the highest priority score (the root element).
    1. It first checks if the heap is empty; if so, it raises an IndexError.
    2. The entry at the root (index 0) is stored temporarily to be returned.
    3. The last element currently in the heap (at index count - 1) is moved to the root position (index 0).
    4. The original position of the last element is set to None, and the count is decremented.
    5. The _trickle_down method is called starting from the root (index 0). This method compares the new root element with its children. If a child has a higher priority score, the element is swapped with the child having the largest score. This process continues down the heap until the element is in a position where its score is greater than or equal to both of its children, restoring the Max Heap property.

6. The originally stored root element (highest priority) is returned.

5.4 Complexity Analysis

Let N be the number of items (triage entries) currently in the heap.

- **Space Complexity: O(N)** (or O(Capacity) if considering the fixed maximum size of the underlying array). The heap stores N entries in an array structure.
- **insert: O(log N)**. Placing the new element takes O(1) time. The subsequent _trickle_up operation involves comparisons and potential swaps along a path from a leaf to the root. In a balanced binary heap, the height is proportional to log N, resulting in O(log N) time complexity for trickling up.
- **extract_next: O(log N)**. Accessing the root and replacing it with the last element takes O(1) time. The subsequent _trickle_down operation involves comparisons and potential swaps along a path from the root to a leaf, which also takes O(log N) time due to the heap's height.
- **Peek (Get Max - if implemented): O(1)**. Accessing the highest priority element (the root at index 0) takes constant time.

5.5 Sample Output & Testing

The heap module was tested by adding patients with valid urgencies (1-5) and varying treatment times via Menu Option 20, observing the heap state via Option 22, and extracting patients via Option 21. The internal logging messages provide traces of the heap maintenance operations.

- **Insertion & Priority Calculation Logs (Menu Option 20 for P1, P2, P3):**

```
Enter your choice: 20
--- Add Patient to Triage Queue ---
Enter Patient ID to add to triage: P1
Found Patient: Test Patient A, Urgency: 1 (1=High, 5=Low)
Enter expected treatment time (in minutes): 50

Adding to triage: Priority: 25.00 | Patient: ID: P1, Name: Test Patient A, Urgency: 1, Condition: heart attack, Dur
ation: 50 mins
...Add complete.
```

```
 Enter your choice: 20
 --- Add Patient to Triage Queue ---
 Enter Patient ID to add to triage: P2
 Found Patient: Test Patient B, Urgency: 3 (1=High, 5=Low)
 Enter expected treatment time (in minutes): 20

 Adding to triage: Priority: 53.00 | Patient: ID: P2, Name: Test Patient B, Urgency: 3, Condition: chest pain, Durat
 ion: 20 mins
   HEAP LOG (Trickle Up): Swapping [idx 1] (Prio 53.00) with Parent [idx 0] (Prio 25.00)
 ...Add complete.
```

```
Enter your choice: 20
--- Add Patient to Triage Queue ---
Enter Patient ID to add to triage: P3
Found Patient: Test Patient 3, Urgency: 5 (1=High, 5=Low)
Enter expected treatment time (in minutes): 80

Adding to triage: Priority: 13.50 | Patient: ID: P3, Name: Test Patient 3, Urgency: 5, Condition: coughing, Duratio
n: 80 mins
...Add complete.
```

- **Heap State After Insertions (Menu Option 22):**

```
Enter your choice: 22

--- Current Triage Queue (Heap Array) ---
  Index 0: Priority: 53.00 | Patient: ID: P2, Name: Test Patient B, Urgency: 3, Condition: chest pain, Duration: 20
 mins
  Index 1: Priority: 25.00 | Patient: ID: P1, Name: Test Patient A, Urgency: 1, Condition: heart attack, Duration:
50 mins
  Index 2: Priority: 13.50 | Patient: ID: P3, Name: Test Patient 3, Urgency: 5, Condition: coughing, Duration: 80 m
ins
Total Patients in Queue: 3 | Capacity: 100
-----------------------------------------
```

- **Extraction Log (Menu Option 21):**

```
Enter your choice: 21
--- Triage Next Patient ---

Triaging next patient...
  HEAP LOG (Trickle Down): Swapping [idx 0] (Prio 13.50) with Largest Child [idx 1] (Prio 25.00)
...Triage complete. Next patient: Priority: 53.00 | Patient: ID: P2, Name: Test Patient B, Urgency: 3, Condition: c
hest pain, Duration: 20 mins

===> NEXT PATIENT TO TREAT: Priority: 53.00 | Patient: ID: P2, Name: Test Patient B, Urgency: 3, Condition: chest p
ain, Duration: 20 mins <===
```

- **Heap State After Extraction (Menu Option 22):**

```
 0. Exit
Enter your choice: 22

--- Current Triage Queue (Heap Array) ---
  Index 0: Priority: 25.00 | Patient: ID: P1, Name: Test Patient A, Urgency: 1, Condition: heart attack, Duration:
50 mins
  Index 1: Priority: 13.50 | Patient: ID: P3, Name: Test Patient 3, Urgency: 5, Condition: coughing, Duration: 80 m
ins
Total Patients in Queue: 2 | Capacity: 100
-----------------------------------------
```

6. Module 4: Sorting Patient Records

6.1 Implementation Details

Module 4 addresses the requirement to sort patient records based on their treatment_duration for reporting and to analyse the performance characteristics of different sorting algorithms. The implementations reside in module4_sorts.py.

- **Merge Sort:**
    - **Implementation:** A recursive, top-down approach is implemented in the mergeSort function.
    - **Logic:** It adheres to the classic divide-and-conquer strategy. The input list (a copy is made initially to preserve the original) is recursively split into halves until sub-lists contain zero or one element (the base case, which is inherently sorted). The sorted sub-lists are then merged back together by the _merge helper function.
    - **_merge Function:** This function takes two already sorted sub-lists as input. It iterates through both lists simultaneously, comparing the treatment_duration attribute of the elements at the current pointers in each list. The element with the smaller duration is appended to a new merged_list, and the corresponding pointer is advanced. Once one list is exhausted, all remaining elements from the other list are appended to the merged_list.
    - **Characteristics:** This implementation returns a new, sorted list, ensuring the original list passed to mergeSort remains unchanged. As is characteristic of Merge Sort, it requires O(N) additional space complexity primarily for the temporary lists created during the merging process.
- **Quick Sort:**
    - **Implementation:** A recursive, in-place sorting algorithm implemented via the quickSort wrapper function which calls the _quickSortRecurse helper.
    - **Partitioning:** The core of the algorithm is the _doPartitioning function. It takes a sub-array (defined by left_idx and right_idx) and a chosen pivot_idx. It partitions this sub-array such that all elements with a treatment_duration less than the pivot's duration are moved to its left, and all elements with a duration greater than or equal are moved to its right. This specific implementation uses a variation similar to the Lomuto scheme: the pivot is first swapped to the end, then an index currIdx tracks the boundary of the "less than" partition while iterating through the list, swapping smaller elements towards the front. Finally, the pivot is swapped into its correct sorted position (at currIdx). The function returns the pivot's final index.

- **Pivot Strategies:** Crucially, the implementation allows for different pivot selection strategies, passed via the pivot_mode argument to quickSort:
    - **left (Naive)**: Selects the first element (left_idx) of the current sub-array. Simple but known to perform poorly on ordered data.
    - **median3 (Default):** Calculates the indices of the first, middle, and last elements of the sub-array. It compares their treatment_duration values and selects the index of the element whose duration is the median of the three. Implemented in the _median_of_three helper. This is a common strategy to mitigate the risk of worst-case behaviour.
    - **random:** Selects a random index between left_idx and right_idx (inclusive) using random.randint. Implemented in the _random_pivot helper. This also effectively avoids worst-case behaviour on average.
- **Recursion:** _quickSortRecurse first selects the pivot based on the chosen strategy, then calls _doPartitioning. After partitioning, it makes recursive calls to sort the sub-arrays to the left and right of the pivot's final position.
- **Error Handling:** The quickSort wrapper function includes a try...except RecursionError block. This anticipates the potential for stack overflow when using the left pivot strategy on large, already sorted or reverse-sorted lists (due to O(N) recursion depth). If caught, an informative error message is printed, and the exception is re-raised so it can be handled by the benchmarking code.

These sorting implementations adapt concepts from COMP1002 Practical 9, specifically modified to operate on PatientRecord objects and sort by the treatment_duration attribute as required by the assignment specification.

6.2 Performance Benchmark Methodology & Results

To evaluate and compare the performance of the implemented sorting algorithms under various conditions, a systematic benchmark was conducted using the run_performance_comparison function located within the main hospital_system.py script.

- **Methodology:**
    - **Algorithms Tested:** The benchmark included:
        1. mergeSort
        2. quickSort with pivot_mode='left'
        3. quickSort with pivot_mode='median3'
        4. quickSort with pivot_mode='random'
    - **Datasets:** For each test scenario, lists of PatientRecord objects were dynamically generated using the _generate_test_patient_list helper function. This function creates records with unique IDs and random (1-5) urgencies but specifically arranges the treatment_duration attribute according to the test condition.

- **Sizes:** Three distinct dataset sizes were used: N = 100, N = 500, and N = 1000 records.
- **Input Conditions**: For each size, performance was measured against three input conditions based on the initial order of treatment_duration:
    1. **random:** Durations in a pseudo-random, unsorted sequence.
    2. **sorted:** Durations arranged in non-decreasing (ascending) order.
    3. **reverse:** Durations arranged in non-increasing (descending) order.
- **Timing:** Execution time was measured using Python's timeit.Timer. For each specific algorithm run, the statement to be timed (e.g., mergeSort(list_copy)) was executed number=1 time per measurement.
- **Repetitions and Averaging:** To obtain more reliable results and mitigate the impact of system fluctuations, each timing measurement (one algorithm on one specific list copy) was repeated REPEATS = 3 times. The average execution time across these 3 repetitions was calculated and reported for each scenario (Algorithm, Size, Condition).
- **List Copies:** Critically, before each individual timing run (within the repetition loop), a fresh copy of the original_list for the current scenario was created (list_copy = list(original_list)). This ensured that in-place algorithms like Quick Sort did not affect subsequent runs and that Merge Sort (which returns a new list) was always operating on the same initial data structure for timing comparison.
- **Error Handling:** The timing loop incorporates a try...except RecursionError block specifically for Quick Sort. If this error occurs (expected for the 'left' pivot), the result is recorded as "RecursionError", and the repetition loop for that scenario terminates early. Other potential exceptions during timing are caught and reported as "TimingError".

- **Benchmark Results Table:**

```
Enter your choice: 32

--- Running Sorting Performance Comparison ---
(Timing based on average of 3 runs per scenario)

Generating random data (Size: 100)...
  Timing Merge Sort...
  Timing Quick Sort (Left)...
  Timing Quick Sort (Median3)...
  Timing Quick Sort (Random)...

Generating sorted data (Size: 100)...
  Timing Merge Sort...
  Timing Quick Sort (Left)...
  Timing Quick Sort (Median3)...
  Timing Quick Sort (Random)...

Generating reverse data (Size: 100)...
  Timing Merge Sort...
  Timing Quick Sort (Left)...
  Timing Quick Sort (Median3)...
  Timing Quick Sort (Random)...

Generating random data (Size: 500)...
  Timing Merge Sort...
  Timing Quick Sort (Left)...
  Timing Quick Sort (Median3)...
  Timing Quick Sort (Random)...

Generating sorted data (Size: 500)...
  Timing Merge Sort...
  Timing Quick Sort (Left)...
```

```
Generating reverse data (Size: 500)...
  Timing Merge Sort...
  Timing Quick Sort (Left)...
  Timing Quick Sort (Median3)...
  Timing Quick Sort (Random)...

Generating random data (Size: 1000)...
  Timing Merge Sort...
  Timing Quick Sort (Left)...
  Timing Quick Sort (Median3)...
  Timing Quick Sort (Random)...

Generating sorted data (Size: 1000)...
  Timing Merge Sort...
  Timing Quick Sort (Left)...

ERROR: Maximum recursion depth exceeded during Quick Sort (pivot='left').
This commonly occurs with 'left' pivot on already sorted or reverse-sorted large datasets.
  Timing Quick Sort (Median3)...
  Timing Quick Sort (Random)...

Generating reverse data (Size: 1000)...
  Timing Merge Sort...
  Timing Quick Sort (Left)...

ERROR: Maximum recursion depth exceeded during Quick Sort (pivot='left').
This commonly occurs with 'left' pivot on already sorted or reverse-sorted large datasets.
  Timing Quick Sort (Median3)...
  Timing Quick Sort (Random)...

--- Performance Comparison Results (Average Time in Seconds) ---
Algorithm                | Size   | Condition  | Avg Time (s)
-----------------------------------------------------------------
```

```
Quick Sort (Left)         | 100    | random     | 0.000382
Quick Sort (Median3)      | 100    | random     | 0.000368
Quick Sort (Random)       | 100    | random     | 0.000427
Merge Sort                | 100    | sorted     | 0.000480
Quick Sort (Left)         | 100    | sorted     | 0.001474
Quick Sort (Median3)      | 100    | sorted     | 0.000322
Quick Sort (Random)       | 100    | sorted     | 0.000409
Merge Sort                | 100    | reverse    | 0.000505
Quick Sort (Left)         | 100    | reverse    | 0.001938
Quick Sort (Median3)      | 100    | reverse    | 0.000408
Quick Sort (Random)       | 100    | reverse    | 0.000422
Merge Sort                | 500    | random     | 0.003307
Quick Sort (Left)         | 500    | random     | 0.002067
Quick Sort (Median3)      | 500    | random     | 0.001634
Quick Sort (Random)       | 500    | random     | 0.001764
Merge Sort                | 500    | sorted     | 0.001354
Quick Sort (Left)         | 500    | sorted     | 0.012895
Quick Sort (Median3)      | 500    | sorted     | 0.000568
Quick Sort (Random)       | 500    | sorted     | 0.000767
Merge Sort                | 500    | reverse    | 0.000740
Quick Sort (Left)         | 500    | reverse    | 0.010724
Quick Sort (Median3)      | 500    | reverse    | 0.000651
Quick Sort (Random)       | 500    | reverse    | 0.000579
Merge Sort                | 1000   | random     | 0.001757
Quick Sort (Left)         | 1000   | random     | 0.001260
Quick Sort (Median3)      | 1000   | random     | 0.001157
Quick Sort (Random)       | 1000   | random     | 0.001381
Merge Sort                | 1000   | sorted     | 0.001264
Quick Sort (Left)         | 1000   | sorted     | RecursionError
Quick Sort (Median3)      | 1000   | sorted     | 0.000931
Quick Sort (Random)       | 1000   | sorted     | 0.001309
Merge Sort                | 1000   | reverse    | 0.001293
Quick Sort (Left)         | 1000   | reverse    | RecursionError
```

6.3 Analysis & Discussion of Benchmark Results

The benchmark results clearly illustrate the performance characteristics of Merge Sort and the different Quick Sort pivot strategies:

- **Performance on Random Data:** As expected, all four algorithms exhibit O(N log N) behaviour on random data, with execution times scaling reasonably well with increasing input size (N). Quick Sort, particularly with the Median-of-Three pivot, often shows slightly better performance than Merge Sort for random inputs in these tests, potentially due to lower constant factors and the benefits of in-place sorting reducing memory overhead.
- **Merge Sort's Consistency:** Merge Sort's performance is highly consistent across random, sorted, and reverse-sorted inputs. Its execution time does not degrade significantly when faced with ordered data, confirming its reliable O(N log N) time complexity in all cases.

This makes it a robust choice when the input data order is unknown or potentially non-random.

- **Quick Sort (Left Pivot) Worst-Case:** The Quick Sort (Left) implementation demonstrates the algorithm's well-known O(N^2) worst-case behaviour. While fast on random data, its performance dramatically worsens on sorted and reverse-sorted lists. For N=500, it becomes significantly slower (approx. 15-18 times slower than Median3/Random). For N=1000, it fails completely due to exceeding Python's default recursion depth limit, resulting in a RecursionError. This failure occurs because choosing the first (or last) element as the pivot on ordered data consistently leads to the most unbalanced partitions possible (one partition with 0 elements, the other with N-1), degrading the recursion depth and overall time complexity to O(N^2).
- **Effectiveness of Median-of-Three and Random Pivots:** Both the Quick Sort (Median3) and Quick Sort (Random) strategies successfully prevent the O(N^2) worst-case scenario. Their execution times on sorted and reverse-sorted data remain comparable to their performance on random data, successfully maintaining the desired O(N log N) average-case efficiency. This highlights the critical importance of choosing a good pivot (one likely to be near the median) in Quick Sort implementations to avoid performance degradation. The Median-of-Three strategy often performs slightly better and more consistently than the Random pivot in these tests, as it provides a deterministic (though slightly more computationally intensive) way to select a pivot likely closer to the actual median, leading to more balanced partitions on average.
- **Practical Implications:** These results underscore the critical importance of pivot selection in Quick Sort. While potentially faster on average than Merge Sort due to its in-place nature, Quick Sort requires a robust pivot strategy (like Median-of-Three or Random) to be reliable in practice, especially when dealing with data that might not be randomly ordered. Merge Sort offers better predictability and guarantees O(N log N) performance, albeit with higher space requirements.

6.4 Complexity Analysis

- **Merge Sort:**
    - **Time Complexity: O(N log N)** (Best, Average, Worst). Merge Sort consistently divides the list and merges, irrespective of the initial data order.
    - **Space Complexity: O(N)**. Requires auxiliary space proportional to the input size for the merging process (both for the recursive calls and the temporary merged list).
- **Quick Sort:**
    - ***Time Complexity (*Average/Best*): O(N log N)***. Achieved when the pivot selection leads to reasonably balanced partitions (typically with Median-of-Three or Random pivots).
    - **Time Complexity (Worst): O(N^2)**. Occurs when pivot selection consistently leads to highly unbalanced partitions (e.g., choosing the smallest or largest element, as demonstrated by the 'Left' pivot on sorted data).

- **Space Complexity (Average): O(log N)**. Primarily due to the depth of the recursion call stack when partitions are balanced.
- **Space Complexity (Worst): O(N)**. Occurs when partitions are unbalanced, leading to a recursion depth proportional to N (as seen with the 'Left' pivot leading to RecursionError).

6.5 Sample Output

- **Benchmark Results Table:** The full table generated by run_performance_comparison (Menu Option 32), demonstrating the performance comparison across algorithms, sizes, and conditions, including the RecursionError, is provided in Section 6.2 above.
- **Live Sort Output Example (Menu Option 30 or 31):** After adding test patients P1 (Duration: 50), P2 (Duration: 20), and P3 (Duration: 80) to the system (which already contained 20 patients loaded from CSV with Duration: 0), running either Merge Sort or Quick Sort yields the following output, correctly sorted by treatment_duration in ascending order:

```
Enter your choice: 30

--- Sorting Current Patients (Merge Sort by Duration) ---
Sorting 23 patients...
Sorted List (by Treatment Duration Ascending):
  ID: P10010, Name: Laura Thomas, Urgency: 9, Condition: Allergic Reaction, Duration: 0 mins
  ID: P10011, Name: Robert Garcia, Urgency: 4, Condition: Fever, Duration: 0 mins
  ID: P10012, Name: Linda Rodriguez, Urgency: 7, Condition: Back Pain, Duration: 0 mins
  ID: P10013, Name: James Hernandez, Urgency: 9, Condition: Concussion, Duration: 0 mins
  ID: P10014, Name: Maria Lopez, Urgency: 5, Condition: Stomach Flu, Duration: 0 mins
  ID: P10015, Name: William Gonzalez, Urgency: 10, Condition: Seizure, Duration: 0 mins
  ID: P10016, Name: Karen Moore, Urgency: 8, Condition: Pneumonia, Duration: 0 mins
  ID: P10017, Name: Paul Allen, Urgency: 7, Condition: Kidney Stone, Duration: 0 mins
  ID: P10018, Name: Nancy Young, Urgency: 10, Condition: Stroke, Duration: 0 mins
  ID: P10019, Name: Mark King, Urgency: 5, Condition: Concussion, Duration: 0 mins
  ID: P10001, Name: John Smith, Urgency: 8, Condition: Broken Arm, Duration: 0 mins
  ID: P10002, Name: Sarah Jones, Urgency: 10, Condition: Chest Pains, Duration: 0 mins
  ID: P10003, Name: David Lee, Urgency: 5, Condition: Sprained Ankle, Duration: 0 mins
  ID: P10004, Name: Emily White, Urgency: 9, Condition: Severe Burn, Duration: 0 mins
  ID: P10005, Name: Michael Brown, Urgency: 7, Condition: Laceration, Duration: 0 mins
  ID: P10006, Name: Jessica Davis, Urgency: 3, Condition: Common Cold, Duration: 0 mins
  ID: P10007, Name: Chris Wilson, Urgency: 10, Condition: Difficulty Breathing, Duration: 0 mins
  ID: P10008, Name: Amanda Taylor, Urgency: 6, Condition: Migraine, Duration: 0 mins
  ID: P10009, Name: Daniel Martinez, Urgency: 8, Condition: Appendicitis, Duration: 0 mins
  ID: P10020, Name: Betty Wright, Urgency: 9, Condition: Sepsis, Duration: 0 mins
  ID: P2, Name: Test Patient B, Urgency: 3, Condition: chest pain, Duration: 20 mins
  ID: P1, Name: Test Patient A, Urgency: 1, Condition: heart attack, Duration: 50 mins
  ID: P3, Name: Test Patient 3, Urgency: 5, Condition: coughing, Duration: 80 mins

Merge Sort completed in 0.000027 seconds.
--------------------------------------------------
```

```
Enter your choice: 31

--- Sorting Current Patients (Quick Sort by Duration) ---
Sorting 23 patients using Quick Sort (Median-of-Three)...
Sorted List (by Treatment Duration Ascending):
  ID: P10002, Name: Sarah Jones, Urgency: 10, Condition: Chest Pains, Duration: 0 mins
  ID: P10011, Name: Robert Garcia, Urgency: 4, Condition: Fever, Duration: 0 mins
  ID: P10003, Name: David Lee, Urgency: 5, Condition: Sprained Ankle, Duration: 0 mins
  ID: P10010, Name: Laura Thomas, Urgency: 9, Condition: Allergic Reaction, Duration: 0 mins
  ID: P10004, Name: Emily White, Urgency: 9, Condition: Severe Burn, Duration: 0 mins
  ID: P10013, Name: James Hernandez, Urgency: 9, Condition: Concussion, Duration: 0 mins
  ID: P10005, Name: Michael Brown, Urgency: 7, Condition: Laceration, Duration: 0 mins
  ID: P10015, Name: William Gonzalez, Urgency: 10, Condition: Seizure, Duration: 0 mins
  ID: P10006, Name: Jessica Davis, Urgency: 3, Condition: Common Cold, Duration: 0 mins
  ID: P10017, Name: Paul Allen, Urgency: 7, Condition: Kidney Stone, Duration: 0 mins
  ID: P10007, Name: Chris Wilson, Urgency: 10, Condition: Difficulty Breathing, Duration: 0 mins
  ID: P10019, Name: Mark King, Urgency: 5, Condition: Concussion, Duration: 0 mins
  ID: P10008, Name: Amanda Taylor, Urgency: 6, Condition: Migraine, Duration: 0 mins
  ID: P10014, Name: Maria Lopez, Urgency: 5, Condition: Stomach Flu, Duration: 0 mins
  ID: P10009, Name: Daniel Martinez, Urgency: 8, Condition: Appendicitis, Duration: 0 mins
  ID: P10012, Name: Linda Rodriguez, Urgency: 7, Condition: Back Pain, Duration: 0 mins
  ID: P10001, Name: John Smith, Urgency: 8, Condition: Broken Arm, Duration: 0 mins
  ID: P10018, Name: Nancy Young, Urgency: 10, Condition: Stroke, Duration: 0 mins
  ID: P10020, Name: Betty Wright, Urgency: 9, Condition: Sepsis, Duration: 0 mins
  ID: P10016, Name: Karen Moore, Urgency: 8, Condition: Pneumonia, Duration: 0 mins
  ID: P2, Name: Test Patient B, Urgency: 3, Condition: chest pain, Duration: 20 mins
  ID: P1, Name: Test Patient A, Urgency: 1, Condition: heart attack, Duration: 50 mins
  ID: P3, Name: Test Patient 3, Urgency: 5, Condition: coughing, Duration: 80 mins

Quick Sort (Median-of-Three) completed in 0.000033 seconds.
-----------------------------------------------------
```

7. Reflection

This project provided valuable practical experience in implementing fundamental data structures and algorithms and integrating them into a cohesive, modular system simulating aspects of hospital operations.

- **Key Learnings:**
    - **Data Structure Suitability:** The project reinforced the importance of selecting the appropriate data structure for a given task's performance requirements. The contrast between the O(V^2) Dijkstra's implementation (due to list scanning) and the potential O(E log V) with a proper priority queue highlighted this. Similarly, the O(1) average access time of the hash table proved crucial for patient lookup, while the heap provided the necessary O(log N) efficiency for priority queue operations.
    - **Algorithm Performance & Trade-offs:** The Module 4 benchmark vividly demonstrated the theoretical concepts of algorithm complexity in practice. Observing the O(N^2) failure (via RecursionError) of Quick Sort with a naive 'left' pivot on ordered data, while Median-of-Three and Random pivots maintained O(N log N) performance, was a significant learning point about the practical impact of implementation choices (like pivot strategy). It also highlighted the trade-off between Quick Sort's potential speed and in-place sorting versus Merge Sort's guaranteed O(N log N) performance and stability at the cost of O(N) space.
    - **Modularity and Integration:** Structuring the code into distinct modules and base classes significantly improved organization and readability compared to a monolithic approach. However, it also required careful management of dependencies (import statements) and ensuring correct data flow between modules (e.g., passing PatientRecord objects or lists between the hash table, heap, and sorting functions). Debugging integration points, such as ensuring treatment_duration was updated correctly in the hash table when adding to the heap, was an essential part of the development process.
    - **Implementation from First Principles:** Adhering to the restriction of not using built-in high-level data structures or sorting functions deepened the understanding of how these structures and algorithms actually work internally (e.g., managing collisions in the hash table, maintaining the heap property, implementing the partitioning step in Quick Sort).
- **Design Trade-offs Revisited:**
    - The choice of Linear Probing in the hash table offered simplicity but required robust resizing to manage potential performance degradation due to clustering.
    - The use of a simple list scan in Dijkstra's simplified the implementation but resulted in a less efficient O(V^2) complexity compared to heap-based approaches.
    - Deciding where to define the PatientRecord class (module2_hash_table.py) required considering its use across multiple modules.

- **Challenges:** Implementing the algorithms correctly, particularly Dijkstra's path reconstruction and Quick Sort's partitioning logic, required careful attention to detail and debugging. Ensuring the benchmark in Module 4 accurately timed the algorithms while handling potential errors like RecursionError also presented a challenge. Integrating the modules smoothly, especially handling the different urgency scales between the initial CSV data(1-10 urgency) and Module 3's requirements(1-5 urgency), necessitated careful validation logic within the main application flow.

Overall, the project was a successful exercise in applying theoretical knowledge of data structures and algorithms to a practical simulation, reinforcing concepts of efficiency, modular design, and the importance of analysing algorithm behaviour under different conditions.

8. Limitations & Assumptions

While the implemented system successfully demonstrates the core concepts and fulfills the assignment requirements, several simplifying assumptions and limitations should be noted:

- **Simplified Data Models:**
    - The PatientRecord includes only essential fields. A real hospital system would manage significantly more complex patient data (demographics, medical history, allergies, billing information, etc.).
    - The DSAGraphNode represents departments simply by name. Real-world navigation might involve floor numbers, building wings, specific room types, or capacity constraints.
- **Urgency Scale Discrepancy:**
    - **Assumption:** The core logic for Module 3 (Heap-Based Scheduling) strictly assumes and requires an urgency scale of 1 (high) to 5 (low) for its priority formula (Priority = (6 - U) + 1000 / T) to function as specified.
    - **Limitation/Handling:** The initial patient data loaded from patients.csv contained urgency values ranging from 1 to 10. The system handles this by allowing Module 2 (Hash Table) to store these values as read. However, Module 3's integration logic (in hospital_system.py, Menu Option 20) includes explicit validation. It checks if a patient's stored urgency is within the 1-5 range before attempting to calculate priority and add them to the triage heap. Patients with urgency outside this range (e.g., 6-10 from the CSV) are rejected from triage with an error message. This required adding new patients manually with valid 1-5 urgencies to effectively test Module 3 and the duration-based sorting in Module 4.
- **Static Treatment Time:** The treatment_time (T) used for priority calculation is assumed to be a fixed estimate provided manually by the user when adding a patient to triage. It does not dynamically change based on patient condition or resource availability.

- **Static Graph:** The hospital layout graph is static, loaded at the start. It does not account for temporary closures, changing travel times due to congestion, or elevator wait times. Edge weights (travel times) are assumed to be non-negative, as required by Dijkstra's algorithm.
- **Basic Collision Handling:** The hash table uses Linear Probing. While simple, it is susceptible to primary clustering, which could degrade performance to O(N) in worst-case scenarios, although dynamic resizing mitigates this risk significantly.
- **Fixed Heap Size:** The DSAHeap implementation uses a numpy array with a fixed maximum capacity set at initialization. It does not dynamically resize if the number of patients in triage exceeds this limit, potentially leading to an OverflowError.
- **No Data Persistence:** Changes made during runtime (adding/deleting patients, adding patients to triage) are not saved back to any file. The system state resets each time the program is run, reloading from the original patients.csv.

These limitations and assumptions were necessary to maintain focus on implementing and analysing the core data structures and algorithms as required by the assignment brief.

9. Conclusion

This project successfully achieved the objective of designing, implementing, and analysing a modular hospital resource management system utilising fundamental data structures and algorithms. Each of the four required modules was developed and integrated, demonstrating key functionalities: efficient hospital navigation using graphs (Module 1), rapid patient record lookup via hash tables (Module 2), priority-based emergency scheduling with heaps (Module 3), and performance analysis of sorting algorithms for patient records (Module 4).

The system adhered to a modular design, separating functionalities into distinct Python files, enhancing code clarity and maintainability. Implementations were built from first principles, avoiding restricted built-in functions as specified. Key algorithms like BFS, DFS, Dijkstra's, Merge Sort, and Quick Sort (with multiple pivot strategies) were correctly implemented and tested.

The performance benchmark in Module 4 provided practical validation of theoretical complexities, clearly showing Merge Sort's consistency and the dramatic O(N^2) worst-case failure of Quick Sort with a naive pivot strategy on ordered data, alongside the effectiveness of robust pivot selection (Median-of-Three, Random) in maintaining O(N log N) average performance. Integration testing confirmed the correct data flow between modules, including the handling of differing data assumptions (like the patient urgency scale) through validation.

Despite necessary simplifications inherent in a simulation, the project effectively demonstrates how core data structures and algorithms can be applied to address real-world challenges in optimisation and efficiency, meeting all functional and analytical requirements outlined in the assignment specification.

10. References

- Wikipedia contributors. (2025). Dijkstra's algorithm. In *Wikipedia*, *The Free Encyclopedia*. Retrieved October 27, 2025, from https://en.wikipedia.org/wiki/Dijkstra%27s_algorithm